

Report

Pattern Recognition

Shekhar “Shakes” Chandra

Version 1.64 Final

Creating and maintaining algorithms (open or closed source) either for pattern recognition and analysis, or in general in the current landscape of big data and artificial intelligence (AI), is important for anyone working with data, whether it be an open source developer, a researcher, an engineer, or a data/computer scientist.

In this report, you will choose a recognition problem from a list provided and solve it with the concepts and materials taught in this course. The problems (and associated data) provided are very close to real research problems that are still being solved! It is hoped that you will get an opportunity to have a go at real-world data crunching and pattern analysis by creating your own solutions to these problems. This will give you an opportunity to practice all the necessary skills required to develop and maintain your own (open source) project(s) and writing a mini-report in the form of a read-me file and a GitHub pull-request. These are skills that are highly desirable in many employment opportunities involving machine learning, big data and artificial intelligence.

You will also have an opportunity to learn how to collaborate on an open source project by forking a pre-existing repository and adding your project to it via a pull-request. This is a common requirement of any medium to large organisations doing AI or data related project/product. You will then be required to write the documentation and instructions for that algorithm to enable others to use it. You will be required to complete the necessary software engineering tasks to complete the report with tasks such as design, commenting, pull requests etc. for this purpose. Your algorithm will directly contribute to the [open source PatternAnalysis library on GitHub](#). Upon submission, the teaching team will provide a professional code review of your submission, so that you get an idea of what to expect when doing such a submission at a work place or for an open-source project.

1 The Report

1.1 Purpose of the Assessment

It is now common place for start-ups, industry and research to use open source (and particularly source versioning) workflows to develop and solve problems that require any form of pattern recognition and artificial intelligence. This is primarily because such solutions require algorithms, but also because it is dominated by deep learning, which is almost entirely based on open-source software and its principles.

This assessment attempts to simulate a real-world scenario when joining a team to perform algorithms research and development. Once completed, it will provide you with the minimal professional development required to work in a small team using Git and deep learning. The teaching team are experts in the models and projects below and will provide valuable feedback to enable you to grow into the field.

1.2 Objective

Your objective is to solve a recognition problem of your choice (see below), document your solution and code, then submit it for code review and acceptance into the [course repository](#). The problems are divided into three difficulty levels where:

- [\[Easy Difficulty\]](#) is capped at 10/20 marks for the implementation portion of the marking criteria
- [\[Normal Difficulty\]](#) is capped at 15/20 marks for the implementation portion marking criteria
- [\[Hard Difficulty\]](#) is not capped for the implementation portion marking criteria

If you are unsure of the difficulty of the task to attempt and the choice to make, see Section [1.4](#) for a recommendation.

1.3 Projects Available

The following problems (and associated requirements) are available with the difficulties indicated noting that those marked for updating may change the dataset of model required in the initial week of release:

The tasks/projects below are finalised. Please check Blackboard for latest version of this file.

1. Segment the 2D OASIS brain data set with an Improved UNet [\[1\]](#) or 2D CAN [\[2\]](#) with all labels having a minimum [Dice similarity coefficient](#) of 0.9 on the test set. [\[Easy Difficulty\]](#)
2. Segment the [HipMRI Study on Prostate Cancer](#) (see Appendix for link) using the processed 2D slices (2D images) available on Rangpur with the 2D UNet [\[3\]](#) with all labels having a minimum [Dice similarity coefficient](#) of 0.75 on the test set on the prostate label. You will need to load Nifti file format and sample code is provided in Appendix [B](#). [\[Easy Difficulty\]](#)
3. Segment the [HipMRI Study on Prostate Cancer](#) (see Appendix for link) using the processed 2D slices (2D images) available on Rangpur with the 2D Improved UNet [\[1\]](#) or 2D CAN [\[2\]](#) with all labels having a minimum [Dice similarity coefficient](#) of 0.75 on the test set on the prostate label. You will need to load Nifti file format and sample code is provided in Appendix [B](#). [\[Normal Difficulty\]](#)
4. Create a suitable multi-layer graph neural network (GNN) model to carry out a semi supervised multi-class node classification using [Facebook Large Page-Page Network dataset](#) [\[4\]](#) with reasonable accuracy. (You may use [this](#) partially processed dataset where the features are in the form of 128 dim vectors.) You should also include a [TSNE](#) or [UMAP](#) embeddings plot with ground truth in colors and provide a brief interpretation/discussion. [\[Normal Difficulty\]](#)
5. Detect lesions within the [ISIC 2017/8 data set](#) (see Appendix for link) with the or a detection network such as the YOLOv7 or newer (pre-trained models are allowed) with all detections having a minimum [Intersection Over Union](#) of 0.8 on the test set and a suitable accuracy for classification. [\[Normal Difficulty\]](#)
6. Create a generative model of the [HipMRI Study on Prostate Cancer](#) (see Appendix for link) using the processed 2D slices (2D images) available on Rangpur using a DCGAN [\[5\]](#) or WGAN [\[6\]](#) (recommended) that has a “reasonably clear image” and a [Structured Similarity \(SSIM\)](#) of over 0.6. [\[Normal Difficulty\]](#)
7. Segment the (downsampled) [Prostate 3D data set](#) (see Appendix for link) with the 3D Improved UNet3D [\[1\]](#) with all labels having a minimum [Dice similarity coefficient](#) of 0.7 on the test set. See also CAN3D [\[7\]](#) for more details and use the data augmentation library [here for TF](#) or use the appropriate

transforms in PyTorch. You may begin with the original 3D UNet [8]. You will need to load Nifti file format and sample code is provided in Appendix B. [Normal Difficulty - 3D UNet] [Hard Difficulty - 3D Improved UNet]

8. Classify Alzheimer’s disease (normal and AD) of the ADNI brain data (see Appendix for link) using one of the latest vision models such as the ConvNeXt [9] or GFNet [10] set having a minimum accuracy of 0.8 on the test set. [Hard Difficulty]
9. Create a classifier based on Siamese network [11] to classify the ISIC 2020 Kaggle Challenge data set (normal and melanoma) having an accuracy of around 0.8 on the test set (see Appendix for links). [Hard Difficulty]
10. Create a generative model of the HipMRI Study on Prostate Cancer using the processed 2D slices (2D images) available here with the using a VQVAE [12] or VQVAE2 [13] that has a “reasonably clear image” and a Structured Similarity (SSIM) of over 0.6. [Hard Difficulty]
11. Create a generative model of one of the ADNI brain data set (see Appendix for links) using either a variant of StyleGAN [14]/StyleGAN2 [15] or Stable Diffusion [16] that has a “reasonably clear image”. You should also include a TSNE or UMAP embeddings plot with ground truth in colors and provide a brief interpretation/discussion. See recent UQ work for hints of how to incorporate labels in style [17]. [Hard Difficulty]

Note: You are likely to need some ‘heavy’ GPU resources.

12. Fine tune a Mamba pretrained state space model to convert grey scale images to color. You can use the this colab notebook as a starting point and refer "MambaIRv2: Attentive State Space Restoration" [18] for the explanation. You will be given a sample test set of images that needs to be ‘reasonably’ colored. Your report/repo should have clear instructions on how to fine tune the models with required code/scripts and setup instructions for full reproducibility. [Hard Difficulty]

Note: You are likely to need some ‘heavy’ GPU resources and the use of torch.run for distributed fine tuning.

13. Fine tune a pretrained encoder-decoder LLM such as T5 [19] or FLAN-T5 [20], or decoder-only LLM such as GPT-2 [21], to translate expert radiology reports into layperson summaries using the BioLay-Summ dataset [Subtask 2.1 of ACL 2025 BioLaySumm workshop]. Evaluate on a held-out test split using ROUGE Scores (rouge1, rouge2, rougeL, rougeLsum). In your report, include model and parameter count, fine-tuning strategy (full vs parameter-efficient such as LoRA), GPU type, VRAM, epochs, and total training time. Also, include 3–5 representative input–output examples with a short error analysis paragraph. [Hard Difficulty]

Note: You are likely to need some ‘heavy’ GPU resources and the use of torch.run for distributed fine tuning.

14. Train a generative time-series model such as TimeGAN to generate synthetic sequences of limit order book (LOB) events using the LOBSTER dataset (Use AMZN level 10 data). Evaluate on a held-out test split using the following metrics:

- **Distribution similarity:** KL divergence ≤ 0.1 between the generated and real *spread* and *midprice return* distributions.
- **Visual similarity:** SSIM > 0.6 between heatmaps of generated vs real LOB depth snapshots.

In your report, include model architecture and parameter count, training strategy (full vs variants such as adversarial-only or supervised-only losses), GPU type, VRAM, epochs, and total training time. Also, include 3–5 representative heatmap visualizations of generated vs real order books with a short error analysis paragraph discussing where the synthetic LOBs succeed and fail. [Hard Difficulty]

Note: You are likely to need some 'heavy' GPU resources and the use of `torch.run` for distributed training. You may also need to do your own reading on how to order books, volatility clustering, and how stock markets work.

A listing and links to the various data sets is provided in Appendix A. Additional recognition problems may become available and will be added to this list. You are also able to propose your own network for the above problems or an entirely different problem to the course coordinator for approval. The teaching staff will decide what constitutes “reasonably clear image” and the SSIM is not strict, but both will be negotiable with the staff depending on the difficulty of the problem.

1.4 Recommended Difficulty Pathway

If you are unsure where to start or unsure which difficulty is best suited for you, I have the following recommendation that will allow you to start, then gradually increase the difficulty by adding parts to the solution to improve the total marks obtainable. The recommended pathway is to begin with the Project 1 on 2D segmentation with the OASIS data, this will give you Easy difficulty marks. Then if time permits, either change the data to The HipMRI 2D data or update your model with to do segmentation in 3D, depending on the model you can get Normal difficulty masks or even hard difficulty.

1.5 Use of Artificial Intelligence

This lab is designed to incorporate the use of AI models to assist your learning experiences. Models such as GitHub Co-Pilot (available for free to students), Microsoft Co-Pilot, Google Gemini (now free for public use), OpenAI ChatGPT (or equivalent) or Claude will be permitted as described in the instructions below. The goal is to show how these models can be useful in understanding and improving your learning and workflows, but to understand their limitations and common problems with their use as well.

2 Submission

The submission of the final report will be done via a GitHub pull-request and a Turn-it-in submission. The pull request should contain

1. the code of your algorithm and test script(s) written to run the algorithm (20 Marks),
2. commit log to provide evidence of originality (5 Marks, Pass Hurdle)
3. documentation via a README.md file (10 Marks)
4. pull-request, with clear description, completed properly (5 Marks, Pass Hurdle)

The Turn-it-in submission should be a PDF version of the README.md file. **You are required to make the pull request to receive any marks for the assessment. You must complete a pull-request (with a working version of the algorithm committed) at least once before the assessment deadline.** The following sections describe the individual tasks to be completed in detail.

3 Checklist

The following steps should be followed to complete this assessment task:

1. fork the [PatternAnalysis library on GitHub](#)

2. checkout the topic-recognition branch within your fork
3. choose a problem via section 1 and create a folder for your problem in the 'recognition' folder of the PatternAnalysis trunk with a meaningful name. We suggest using a name that incorporates the name of your model you are implementing appended with your name or ID
4. create a README.md file that describes your report.
5. complete the solution to the recognition problem by building your model, making sure to commit any progress to the topic-recognition branch of your fork
6. Do not commit datasets or model files, your fork and commits should only have code/scripts plus any additional content needed for your README.md file
7. complete the test driver script that calls and runs your algorithm
8. complete the documentation of your algorithm including a summary of your results and model in your README.md file
9. lodge a pull request
10. complete a Turn-it-in submission of the PDF of the README.md file.

4 Recognition Problem (20 Marks)

Your implementation must include the following files

1. **"modules.py"** containing the source code of the components of your model. Each component must be implemented as a class or a function
2. **"dataset.py"** containing the data loader for loading and preprocessing your data
3. **"train.py"** containing the source code for **training, validating, testing** and **saving** your model. The model should be imported from "modules.py" and the data loader should be imported from "dataset.py". Make sure to plot the losses and metrics during training
4. **"predict.py"** showing example usage of your **trained** model. Print out any results and / or provide visualisations where applicable
5. **"README.MD"** to sufficiently document your project (see Section 6).

Note: You may create other helper files such as "utils.py" to better organise your project. You may reuse some of your own elements from a previous demo, but the final model must be substantially different from your previous work.

Marking Criteria

1. Algorithm solves the respective pattern recognition problem appropriately (5 Marks)
2. Algorithm implemented in TF/PyTorch demonstrated and functions as intended to solve the respective problem (3 Marks)
3. Good design implemented in TF/PyTorch that solves the problem (1 Marks)

4. Commenting (1 Mark)
5. Algorithm is normal or above difficulty while implemented in TF/PyTorch while solving the respective problem (5 Marks)
6. Algorithm is hard difficulty while implemented in TF/PyTorch while solving the respective problem (5 Marks)

5 Commit Log (5 Marks, Pass Hurdle)

Requirements

1. **Ensure that when you are implementing your algorithm to solve the recognition problem, you are committing regularly and keeping a detailed commit log.** You should for example commit whenever you reach any small milestone, such as creating a module or a working test script, completed the algorithm and completed the documentation etc.
2. **You must have a reasonable commit log showing progress and development of your algorithm. You should not commit your entire submission as one commit.** You will not receive any marks if your algorithm only contains a single commit. You may not pass the assessment if your commit log does not provide sufficient evidence of individual work and originality.
3. **Do not commit datasets or model files.** Your fork and commits should only have code/scripts plus any additional content needed for your README.md file.

Marking Criteria

1. Meaningful commit messages with evidence of individual work and originality (2 Marks)
2. Progressive commits and logical log structure providing evidence of individual work and originality (3 Marks)

6 Documentation (10 Marks)

Once your algorithm has been implemented, you will need to provide sufficient comments in your code and documentation of your algorithm in a README.md file.

Requirements

1. **The readme file should contain a title, a description of the algorithm and the problem that it solves (approximately a paragraph), how it works in a paragraph and a figure/visualisation.**
2. It should also list any dependencies required, including versions and address reproducibility of results, if applicable.
3. provide example inputs, outputs and plots of your algorithm
4. The read me file should be properly formatted using [GitHub markdown](#)
5. Describe any specific pre-processing you have used with references if any. Justify your training, validation and testing splits of the data.

Marking Criteria

1. description and explanation of the working principles of the algorithm implemented and the problem it solves (5 Marks)
2. description of usage and comments throughout scripts (3 Marks)
3. proper formatting using [GitHub markdown](#) (2 Mark)

7 Pull Request (5 Marks)

Once your algorithm module, test script and read me file are completed, lodge a pull-request to the [Pattern-Analysis repository](#) via the topic-recognition branch. **A pull-request must be made to receive any marks for this assessment. This pull request must have a working version of the algorithm committed.**

You can see a guide on creating pull requests via GitHub [here](#) and a guide about pull-requests via GitHub [here](#). See also [this tutorial by Atlassian](#) that explains the concept quite well.

Marking Criteria

1. Creating a pull-request into correct branch with a working and demonstrable version of the algorithm (2 Marks)
2. Incorporating feedback into the pull request (2 Marks)
3. Description of and comments within the pull request (1 Mark)

8 Tips and Advice

It is best advised to try an Easy problem first especially if you are not so confident with TF/PyTorch and problems provided. Once working, expand up to the Normal problem such as the recommended pathway mentioned in Section 1.4. For generative problems, try to get your code working on a simpler data set such as the MNIST first and then attempt it with the data for your problem. You should see the previous years successful pull requests [here](#) and [here](#), as well as the type of expected content and hints to the problems. **Hard problems should only be attempted if you are extremely confident with your deep learning skills!**

9 Academic Integrity and Misconduct

Academic Integrity is a core value of the UQ community and as such high academic integrity expectations apply to assessments. Using online resources for self-learning is fine, however **simply re-using entire blocks of code for them is not acceptable and may result in a violation of the [Student Charter](#) as a UQ student and an investigation into Academic Misconduct.** You must follow the guidelines for Academic Integrity found [here](#), which also applies to code. Where possible, you should write your own code and if certain elements are required from online resources should be referenced in your pull-request or code. For more information see the [UQ policy here](#) and a tutorial on understanding this issue further go to the [official UQ tutorial](#).

10 Conclusion

By completing this report, you will obtain experience in developing and maintaining your own open source project, how to contribute to other open source projects and good practices in algorithmic design and implementation. All the while, you will become even more knowledgeable in Tensorflow/PyTorch and pattern recognition!

Appendix

A Datasets

The following datasets are all available on the Rangpur HPC in the */home/groups/comp3710* directory. The information and details for Rangpur can be found at the link [here](#). You can also find these datasets at the sources below:

1. OASIS Brain data set (9K images) - This is part of the [OASIS brain study](#) with segmentation labels. The preprocessed version of this data set can be found on Rangpur.

Rangpur Path: `/home/groups/comp3710/OASIS`

2. ISIC 2017/8 dermoscopic imaging challenge data for skin cancer - This is part of the [ISIC 2017/8 challenge](#) and comes with segmentation labels and lesion class labels.

Rangpur Path: `/home/groups/comp3710/ISIC2018`

3. [ISIC 2020 Kaggle Challenge data set](#) with dermoscopic imaging having image-level class labels (normal or melanoma). See also the Kaggle Challenge page [here](#) for more details.
4. The [ADNI dataset for Alzheimer's disease](#). Look at the DOWNLOAD > Image Collections > Advanced Search area to download the data. The preprocessed version of this data set has two classes: Alzheimer's disease (AD) and Cognitive Normal (CN). It can be found on Rangpur at:

Rangpur Path: `/home/groups/comp3710/ADNI`

5. The [HipMRI Study for prostate cancer radiotherapy](#) consisting of Prostate 2D MRI slide data (under the 'keras_slices_data' folder) for 2D tasks and [Prostate 3D MRI data](#) for 3D tasks of 38 patients with 211 3D MRI volumes with MRIs in the 'semantic_MRs' folder and the segmentations in the 'semantic_labels_only'. The body outline, bone, bladder, rectum and prostate regions are segmented in 3D by a MR physicist with more than 10 years of experience.

Rangpur Path: `/home/groups/comp3710/HipMRI_Study_open`

2D Data: `/home/groups/comp3710/HipMRI_Study_open/keras_slices_data`

6. BioLaySumm 2025 ACL workshop, Subtask2.1 LaymanRRG open-source track for layperson translation of radiology reports. Dataset card and downloads: [Hugging Face](#). Task overview: [biolaysumm.org](#).

B Reading Nifti Files

The following code blocks are examples of loading Nifti image files using the [Nilearn](#) and [Nibabel](#) libraries, which are the standard file format in medical imaging. Listing 1 shows how to load 2D Nifti files and Listing 2 shows how to load 3D Nifti files as numpy arrays.

```
1 import numpy as np
2 import nibabel as nib
3 from tqdm import tqdm
4
5 def to_channels(arr: np.ndarray, dtype=np.uint8) -> np.ndarray:
6     channels = np.unique(arr)
7     res = np.zeros(arr.shape + (len(channels),), dtype=dtype)
8     for c in channels:
9         c = int(c)
10        res[..., c:c+1][arr == c] = 1
11
12    return res
13
14 #load medical image functions
15 def load_data_2D(imageNames, normImage=False, categorical=False, dtype=np.float32, \
16    getAffines=False, early_stop=False):
17    '''
18    Load medical image data from names, cases list provided into a list for each.
19
20    This function pre-allocates 4D arrays for conv2d to avoid excessive memory \
21    usage.
22
23    normImage: bool (normalise the image 0.0-1.0)
24    early_stop: Stop loading pre-maturely, leaves arrays mostly empty, for quick \
25    loading and testing scripts.
26    '''
27    affines = []
28
29    #get fixed size
30    num = len(imageNames)
31    first_case = nib.load(imageNames[0]).get_fdata(caching='unchanged')
32    if len(first_case.shape) == 3:
33        first_case = first_case[:, :, 0] #sometimes extra dims, remove
34    if categorical:
35        first_case = to_channels(first_case, dtype=dtype)
36        rows, cols, channels = first_case.shape
37        images = np.zeros((num, rows, cols, channels), dtype=dtype)
38    else:
39        rows, cols = first_case.shape
40        images = np.zeros((num, rows, cols), dtype=dtype)
41
42    for i, inName in enumerate(tqdm(imageNames)):
43        niftiImage = nib.load(inName)
44        inImage = niftiImage.get_fdata(caching='unchanged') #read disk only
45        affine = niftiImage.affine
46        if len(inImage.shape) == 3:
47            inImage = inImage[:, :, 0] #sometimes extra dims in HipMRI_study data
```

```

45     inImage = inImage.astype(dtype)
46     if normImage:
47         #~ inImage = inImage / np.linalg.norm(inImage)
48         #~ inImage = 255. * inImage / inImage.max()
49         inImage = (inImage - inImage.mean()) / inImage.std()
50     if categorical:
51         inImage = utils.to_channels(inImage, dtype=dtype)
52         images[i,:,:,:] = inImage
53     else:
54         images[i,:,:] = inImage
55
56     affines.append(affine)
57     if i > 20 and early_stop:
58         break
59
60 if getAffines:
61     return images, affines
62 else:
63     return images

```

```

1 def load_data_3D(imageNames, normImage=False, categorical=False, dtype=np.float32, \
2     getAffines=False, orient=False, early_stop=False):
3     '''
4     Load medical image data from names, cases list provided into a list for each.
5
6     This function pre-allocates 5D arrays for conv3d to avoid excessive memory \
7     usage.
8
9     normImage: bool (normalise the image 0.0-1.0)
10    orient: Apply orientation and resample image? Good for images with large slice \
11    thickness or anisotropic resolution
12    dtype: Type of the data. If dtype=np.uint8, it is assumed that the data is \
13    labels
14    early_stop: Stop loading pre-maturely? Leaves arrays mostly empty, for quick \
15    loading and testing scripts.
16    '''
17    affines = []
18
19    #~ interp = 'continuous'
20    interp = 'linear'
21    if dtype == np.uint8: #assume labels
22        interp = 'nearest'
23
24    #get fixed size
25    num = len(imageNames)
26    niftiImage = nib.load(imageNames[0])
27    if orient:
28        niftiImage = im.applyOrientation(niftiImage, interpolation=interp, scale=1)
29        #~ testResultName = "oriented.nii.gz"
30        #~ niftiImage.to_filename(testResultName)
31    first_case = niftiImage.get_fdata(caching='unchanged')
32    if len(first_case.shape) == 4:

```

```

28     first_case = first_case[:, :, :, 0] #sometimes extra dims, remove
29 if categorical:
30     first_case = to_channels(first_case, dtype=dtype)
31     rows, cols, depth, channels = first_case.shape
32     images = np.zeros((num, rows, cols, depth, channels), dtype=dtype)
33 else:
34     rows, cols, depth = first_case.shape
35     images = np.zeros((num, rows, cols, depth), dtype=dtype)
36
37 for i, inName in enumerate(tqdm(imageNames)):
38     niftiImage = nib.load(inName)
39     if orient:
40         niftiImage = im.applyOrientation(niftiImage, interpolation=interp,
41                                         scale=1)
42     inImage = niftiImage.get_fdata(caching='unchanged') #read disk only
43     affine = niftiImage.affine
44     if len(inImage.shape) == 4:
45         inImage = inImage[:, :, :, 0] #sometimes extra dims in HipMRI_study data
46     inImage = inImage[:, :, :depth] #clip slices
47     inImage = inImage.astype(dtype)
48     if normImage:
49         #~ inImage = inImage / np.linalg.norm(inImage)
50         #~ inImage = 255. * inImage / inImage.max()
51         inImage = (inImage - inImage.mean()) / inImage.std()
52     if categorical:
53         inImage = utils.to_channels(inImage, dtype=dtype)
54         #~ images[i, :, :, :, :] = inImage
55         images[i, :inImage.shape[0], :inImage.shape[1], :inImage.shape[2], :inImage\
56             .shape[3]] = inImage #with pad
57     else:
58         #~ images[i, :, :, :] = inImage
59         images[i, :inImage.shape[0], :inImage.shape[1], :inImage.shape[2]] = \
60             inImage #with pad
61
62     affines.append(affine)
63     if i > 20 and early_stop:
64         break
65
66 if getAffines:
67     return images, affines
68 else:
69     return images

```

References

- [1] F. Isensee, P. Kickingeder, W. Wick, M. Bendszus, and K. H. Maier-Hein, “Brain Tumor Segmentation and Radiomics Survival Prediction: Contribution to the BRATS 2017 Challenge,” Feb. 2018. [Online]. Available: <https://arxiv.org/abs/1802.10508v1>
- [2] F. Yu and V. Koltun, “Multi-Scale Context Aggregation by Dilated Convolutions,” *arXiv:1511.07122 [cs]*, Apr. 2016, arXiv: 1511.07122. [Online]. Available: <http://arxiv.org/abs/1511.07122>

- [3] O. Ronneberger, P. Fischer, and T. Brox, "U-Net: Convolutional Networks for Biomedical Image Segmentation," in *Medical Image Computing and Computer-Assisted Intervention – MICCAI 2015*, ser. Lecture Notes in Computer Science, N. Navab, J. Hornegger, W. M. Wells, and A. F. Frangi, Eds. Cham: Springer International Publishing, 2015, pp. 234–241.
- [4] B. Rozemberczki, C. Allen, and R. Sarkar, "Multi-scale Attributed Node Embedding," *arXiv:1909.13021 [cs, stat]*, Mar. 2021, arXiv: 1909.13021. [Online]. Available: <http://arxiv.org/abs/1909.13021>
- [5] A. Radford, L. Metz, and S. Chintala, "Unsupervised Representation Learning with Deep Convolutional Generative Adversarial Networks," *arXiv:1511.06434 [cs]*, Jan. 2016, arXiv: 1511.06434. [Online]. Available: <http://arxiv.org/abs/1511.06434>
- [6] M. Arjovsky, S. Chintala, and L. Bottou, "Wasserstein Generative Adversarial Networks," in *Proceedings of the 34th International Conference on Machine Learning*. PMLR, Jul. 2017, pp. 214–223, iSSN: 2640-3498. [Online]. Available: <https://proceedings.mlr.press/v70/arjovsky17a.html>
- [7] W. Dai, B. Woo, S. Liu, M. Marques, C. B. Engstrom, P. B. Greer, S. Crozier, J. A. Dowling, and S. S. Chandras, "CAN3D: Fast 3D Medical Image Segmentation via Compact Context Aggregation," *arXiv:2109.05443 [cs, eess]*, Sep. 2021, arXiv: 2109.05443. [Online]. Available: <http://arxiv.org/abs/2109.05443>
- [8] O. Cicek, A. Abdulkadir, S. S. Lienkamp, T. Brox, and O. Ronneberger, "3D U-Net: Learning Dense Volumetric Segmentation from Sparse Annotation," in *Medical Image Computing and Computer-Assisted Intervention – MICCAI 2016*, ser. Lecture Notes in Computer Science, S. Ourselin, L. Joskowicz, M. R. Sabuncu, G. Unal, and W. Wells, Eds. Cham: Springer International Publishing, 2016, pp. 424–432.
- [9] Z. Liu, H. Mao, C.-Y. Wu, C. Feichtenhofer, T. Darrell, and S. Xie, "A ConvNet for the 2020s," *arXiv:2201.03545 [cs]*, Mar. 2022, arXiv: 2201.03545. [Online]. Available: <http://arxiv.org/abs/2201.03545>
- [10] Y. Rao, W. Zhao, Z. Zhu, J. Zhou, and J. Lu, "GFNet: Global Filter Networks for Visual Recognition," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 45, no. 9, pp. 10 960–10 973, Sep. 2023, conference Name: IEEE Transactions on Pattern Analysis and Machine Intelligence. [Online]. Available: <https://ieeexplore.ieee.org/document/10091201?denied=>
- [11] G. Koch, R. Zemel, R. Salakhutdinov *et al.*, "Siamese neural networks for one-shot image recognition," in *ICML deep learning workshop*, vol. 2. Lille, 2015, p. 0.
- [12] A. v. d. Oord, O. Vinyals, and K. Kavukcuoglu, "Neural Discrete Representation Learning," *arXiv:1711.00937 [cs]*, May 2018, arXiv: 1711.00937. [Online]. Available: <http://arxiv.org/abs/1711.00937>
- [13] A. Razavi, A. v. d. Oord, and O. Vinyals, "Generating Diverse High-Fidelity Images with VQ-VAE-2," *arXiv:1906.00446 [cs, stat]*, Jun. 2019, arXiv: 1906.00446. [Online]. Available: <http://arxiv.org/abs/1906.00446>
- [14] T. Karras, S. Laine, and T. Aila, "A Style-Based Generator Architecture for Generative Adversarial Networks," *arXiv:1812.04948 [cs, stat]*, Mar. 2019, arXiv: 1812.04948. [Online]. Available: <http://arxiv.org/abs/1812.04948>
- [15] T. Karras, S. Laine, M. Aittala, J. Hellsten, J. Lehtinen, and T. Aila, "Analyzing and Improving the Image Quality of StyleGAN," *arXiv:1912.04958 [cs, eess, stat]*, Mar. 2020, arXiv: 1912.04958. [Online]. Available: <http://arxiv.org/abs/1912.04958>

- [16] R. Rombach, A. Blattmann, D. Lorenz, P. Esser, and B. Ommer, "High-Resolution Image Synthesis with Latent Diffusion Models," Apr. 2022, arXiv:2112.10752 [cs]. [Online]. Available: <http://arxiv.org/abs/2112.10752>
- [17] S. Liu, L. Liu, C. Engstrom, X. V. To, Z. Ge, S. Crozier, F. Nasrallah, and S. S. Chandra, "Style-Based Manifold for Weakly-Supervised Disease Characteristic Discovery," in *Medical Image Computing and Computer Assisted Intervention – MICCAI 2023*, ser. Lecture Notes in Computer Science, H. Greenspan, A. Madabhushi, P. Mousavi, S. Salcudean, J. Duncan, T. Syeda-Mahmood, and R. Taylor, Eds. Cham: Springer Nature Switzerland, 2023, pp. 368–378.
- [18] H. Guo, Y. Guo, Y. Zha, Y. Zhang, W. Li, T. Dai, S.-T. Xia, and Y. Li, "Mambairv2: Attentive state space restoration," in *Proceedings of the Computer Vision and Pattern Recognition Conference*, 2025, pp. 28 124–28 133.
- [19] C. Raffel, N. Shazeer, A. Roberts, K. Lee, S. Narang, M. Matena, Y. Zhou, W. Li, and P. J. Liu, "Exploring the limits of transfer learning with a unified text-to-text transformer," *Journal of Machine Learning Research*, vol. 21, no. 140, pp. 1–67, 2020.
- [20] H. W. Chung, L. Hou, S. Longpre *et al.*, "Scaling instruction-finetuned language models," *arXiv preprint arXiv:2210.11416*, 2022. [Online]. Available: <https://arxiv.org/abs/2210.11416>
- [21] A. Radford, J. Wu, R. Child, D. Luan, D. Amodei, and I. Sutskever, "Language models are unsupervised multitask learners," OpenAI, Tech. Rep., 2019. [Online]. Available: https://cdn.openai.com/better-language-models/language_models_are_unsupervised_multitask_learners.pdf